



Operadores e Condicionais

Aula 03 · Bloco 1 — Fundamentos

Decidir com if e switch — e a armadilha das Strings

Nesta aula

1. Operadores e precedência
2. Relacionais e lógicos
3. `if`, `else if`, `else` — e as regras de ouro
4. `switch` clássico e o **moderno**
5. `==` × `.equals()` — a pegadinha mais importante do bloco

Aritméticos e precedência

```
int a = 7, b = 2;
```

```
a / b      // 3   ← divisão inteira (aula 02)
```

```
a % b      // 1   ← resto: "é par?", "é múltiplo?"
```

```
2 + 3 * 4   // 14  – a precedência é a da matemática
```

```
(2 + 3) * 4  // 20  – parênteses mandam em tudo
```

Atalhos que você usará o curso inteiro:

```
contador++;   contador--;   contador += 5;   contador *= 2;
```

Relacionais e lógicos

```
nota > 6      // true
nota == 10    // IGUALDADE é ==, com DOIS sinais
nota != 10    // diferente

temIdade && temDocumento // E: exige as DUAS
temIdade || temDocumento // OU: basta UMA
!temIdade      // NÃO: inverte
```

⚠ **= atribui**, **== compara**. Em Java, `if (nota = 10)` **nem compila** — o `if` exige um `boolean`. O compilador salva sua pele aqui, coisa que em outras linguagens não acontece.

if / else if / else

```
if (nota >= 7) {  
    System.out.println("Aprovado");  
} else if (nota >= 5) {  
    System.out.println("Recuperação");  
} else {  
    System.out.println("Reprovado");  
}
```

- A condição **precisa** ser **boolean** — não existe "0 é falso" em Java;
- Testadas **em ordem**: o primeiro verdadeiro encerra a cadeia.

⚠ Sempre use chaves

Mesmo com uma linha só.

```
if (aprovado)
    System.out.println("Parabéns!");
    System.out.println("Você passou!");
```

A segunda linha imprime **SEMPRE** — ela nunca esteve dentro do `if`.

A indentação enganou você. As chaves não enganariam.

switch clássico

```
switch (diaDaSemana) {  
    case 1:  
        System.out.println("Domingo");  
        break;           // sem o break, "vaza" para o próximo case!  
    case 2:  
        System.out.println("Segunda");  
        break;  
    default: System.out.println("Dia inválido");  
}
```

⚠ Esquecer o `break` é o bug clássico — o famoso *fall-through*.

O `switch` moderno — Java 14+

```
String nomeDoDia = switch (diaDaSemana) {  
    case 1 -> "Domingo";  
    case 2 -> "Segunda";  
    case 3 -> "Terça";  
    default -> "Dia inválido";  
};
```

Não precisa de `break` e ainda **devolve um valor**.

💡 Use `switch` quando testar **um mesmo valor** contra opções fixas. Use `if/else if` quando as condições forem diferentes entre si.



A pegadinha mais importante do bloco

```
String a = "Java";  
String c = new String("Java");
```

<code>a == c</code>	→	false	😱
<code>a.equals(c)</code>	→	true	✅

Mesmo texto. Respostas diferentes.

Por quê?

`String` é **objeto**. Então:

- `==` pergunta: *"são o mesmo objeto na memória?"*
- `.equals()` pergunta: *"têm o mesmo conteúdo?"*

Literais idênticos escritos no código são otimizados para o mesmo objeto — por isso

`"Java" == "Java"` dá `true`, **por acidente**.

Mas texto vindo do `Scanner`, de arquivo ou de concatenação **não é** o mesmo objeto.

Na prática

```
String senha = scanner.nextLine();

if (senha == "1234") { ... }           // ✗ nunca será true
if (senha.equals("1234")) { ... }      // ✓
if ("1234".equals(senha)) { ... }      // ✓✓ e ainda evita erro se for null
if (senha.equalsIgnoreCase("Sim")) { } // ignora maiúsculas
```

🔧 **Regra do curso:** `==` só para primitivos — `int`, `double`, `boolean`, `char`. Para objetos, sempre `.equals()`.

Operador ternário

```
String situacao = (nota >= 7) ? "Aprovado" : "Reprovado";  
//          condição      ? se true : se false
```

Um `if/else` que **devolve valor**, em uma linha.

Ótimo para escolhas curtas. Péssimo para lógica comprida — e nunca aninhe ternário dentro de ternário.

Exercícios da aula

Na pasta `aula-03/` :

1. **FaixaEtaria.java** — criança / adolescente / adulto / idoso, validando negativo;
2. **Calculadora.java** — com **switch moderno**, tratando divisão por zero;
3. **Igualdade.java** — compare com `==` , `.equals()` e `.equalsIgnoreCase()` , e explique o resultado;
4. **Bissexto.java** — teste com 2024  , 1900  , 2000  ;
5. **Desafio**  **Ingresso.java** — meia-entrada, desconto de terça, saída com `printf` .

Próxima aula

Aula 04 — Laços, Arrays e Métodos

Repetir, guardar coleções
e organizar o código em blocos reutilizáveis.